

Ejercicio 1 Gestión de Estudiantes

Ingeniería de Sistemas y Computación

Estudiante

Manuel Eduardo Alarcon Aza

Profesor

William Javier Matallana Porras

Universidad de Cundinamarca – UDEC

Chía

2026 - 1

Fase 1: Levantamiento de Requerimientos

1. Información general del proyecto

Nombre del proyecto	Sistema de Gestión de Pagos
Integrantes	William Alexander Matallana
Programa académico	Ingeniería de Sistemas y Computación
Fecha	10/02/2026
Lenguaje de programación	Java
Tipo de aplicación	Consola

2. Descripción general del sistema

- El sistema permite registrar estudiantes con su información académica básica.
- Cada estudiante tiene un identificador único generado automáticamente, nombre, correo electrónico y programa académico.
- El sistema permite listar, buscar, modificar y eliminar estudiantes registrados.
- La aplicación está desarrollada en Java aplicando el modelo de capas (presentación, negocio, datos).

3. Requerimientos Funcionales (RF)

ID	Nombre	Descripción	Entrada(s)	Proceso	Salida
RF-01	Registrar estudiante	Permite registrar un nuevo estudiante en el sistema	nombre, correo, programaAcademico	Valida campos y genera ID automático	Estudiante registrado con ID asignado
RF-02	Listar estudiantes	Muestra todos los estudiantes registrados	—	Recorre la lista de estudiantes	Tabla con todos los estudiantes
RF-03	Buscar estudiante	Permite buscar un estudiante por su ID	id	Busca en la lista por identificador	Datos del estudiante encontrado

RF-04	Modificar estudiante	Permite actualizar la información de un estudiante existente	id, nombre, correo, programaAcademico	Valida y actualiza los datos	Estudiante modificado exitosamente
-------	----------------------	--	---------------------------------------	------------------------------	------------------------------------

4. Requerimientos No Funcionales (RNF)

ID	Tipo	Descripción
RNF-01	Validación	El sistema no debe permitir registrar estudiantes con campos vacíos o nulos
RNF-02	Validación	El correo electrónico debe tener formato válido (contener @ y dominio con punto)
RNF-03	Generación	El identificador único debe generarse automáticamente mediante UUID y no puede ser ingresado manualmente
RNF-04	Estructura	El sistema debe estar organizado por capas: modelo, datos, negocio y presentación

5. Relación Requerimiento – POO

ID Requerimiento	Clase	Método	Tipo
RF-01	EstudianteServicio	registrar()	Funcional
RF-02	EstudianteServicio	listarTodos()	Funcional
RF-03	EstudianteServicio	buscarPorId()	Funcional
RF-04	EstudianteServicio	modificar()	Funcional
RF-05	EstudianteServicio	eliminar()	Funcional
RNF-01	EstudianteServicio	validarNoVacio()	No Funcional
RNF-02	EstudianteServicio	validarCorreo()	No Funcional
RNF-03	EstudianteServicio	generarId()	No Funcional

Fase 2: Análisis

Entidades identificadas:

Estudiante (id, nombre, correo, programaAcademico)

Relaciones: No hay relaciones entre entidades en este sistema, ya que solo existe una entidad principal “Estudiante”.

Flujos de trabajo

- El usuario registra un estudiante → el sistema valida campos y genera el ID automáticamente.
- El usuario lista estudiantes → el sistema recorre y muestra todos los registros.
- El usuario busca por ID → el sistema localiza y muestra el estudiante.
- El usuario modifica un estudiante → el sistema valida y actualiza los datos.
- El usuario elimina un estudiante → el sistema confirma y elimina el registro.

Diagrama de casos de uso

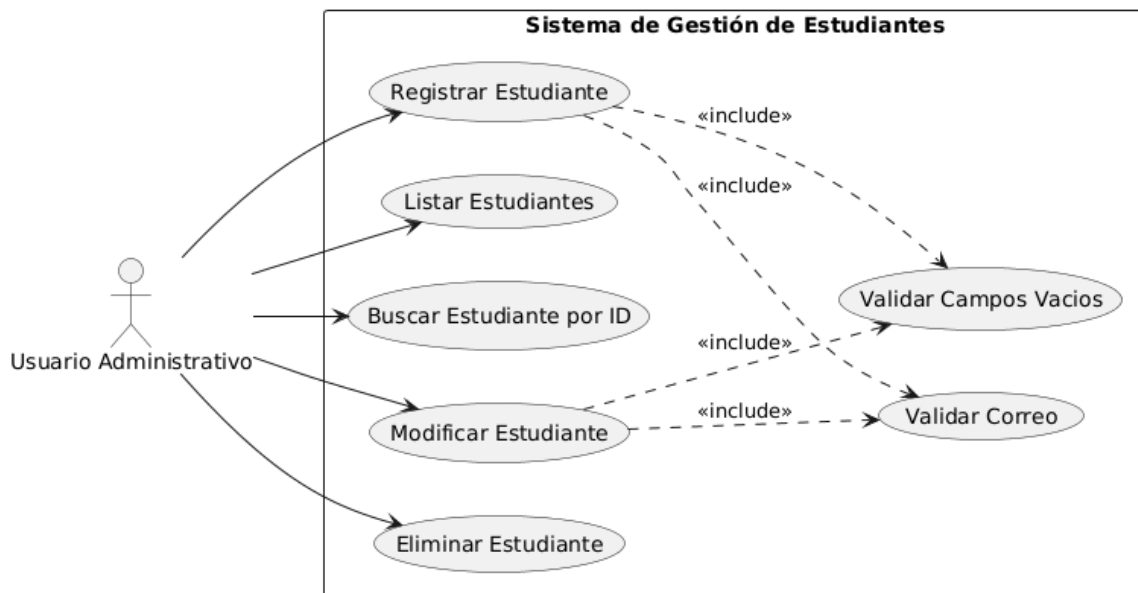


Diagrama de clases UML

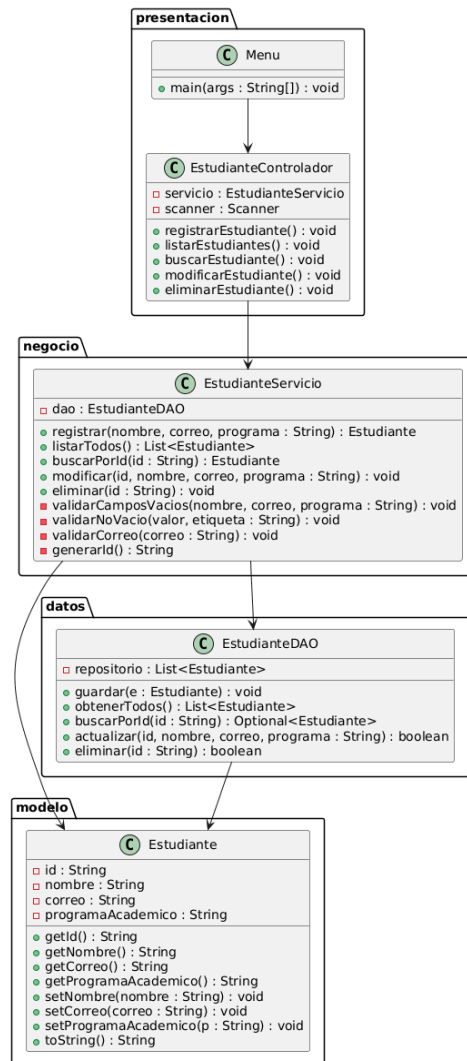


Diagrama de Flujo



Fase 3: Diseño

El sistema está organizado bajo una arquitectura en capas, donde cada capa tiene una responsabilidad específica y se comunica únicamente con la capa inmediatamente inferior. Las cuales son:

- ✓ **Capa de Presentación:** Contiene la clase Main, que es el punto de entrada del sistema. Se encarga de mostrar el menú al usuario, leer las entradas por consola y mostrar los resultados. No contiene lógica de negocio.
- ✓ **Capa de Negocio:** Contiene la clase EstudianteServicio, que aplica todas las reglas del sistema: validar que los campos no estén vacíos, verificar el formato del correo electrónico y generar el identificador único automáticamente.
- ✓ **Capa de Datos:** Contiene la clase EstudianteDAO, que gestiona el almacenamiento de los estudiantes en memoria mediante una lista ArrayList. Es la única capa que accede directamente al repositorio de datos.

Estructura de Paquetes y Clases

El proyecto se organiza en cuatro paquetes dentro de la carpeta src:

- **modelo:** contiene la clase Estudiante.java, que representa la entidad principal del sistema con sus atributos y métodos de acceso.
- **datos:** contiene la clase EstudianteDAO.java, responsable de las operaciones de almacenamiento y recuperación de datos.
- **negocio:** contiene la clase EstudianteServicio.java, responsable de aplicar las validaciones y reglas del negocio antes de delegar a la capa de datos.
- **presentación:** contiene la clase Menu.java, que incluye el método main y gestiona toda la interacción con el usuario por consola.

Tecnologías Utilizadas

- Lenguaje: Java
- Tipo de aplicación: Consola
- Almacenamiento: En memoria mediante ArrayList
- Generación de ID: UUID de la librería estándar de Java

Fase 4: Codificación

Link del repositorio: <https://github.com/Alarconmanuel/gestionEstudiantes.git>